

Decisiones Cap 13

Intro

Regresión lineal simple (RLS)

Predictor (variable independiente)

Variable de salida (variable dependiente)

Relación directa o inversa

Error $y = y^{\wedge} + e$ (error igual a la diferencia entre valor observado y valor esperado)

Los errores se pueden graficar en un gráfico de residuos, donde $y^{\wedge}$ corresponde a una recta y cada valor observado “y” se representa con un punto.

13.1 Correlación

Permite medir la “fuerza” de una relación lineal entre dos variables

$R > 0 \rightarrow$ relación directa

$R < 0 \rightarrow$ relación inversa

En un gráfico de residuos, mientras más “cerca” estén las observaciones a la línea de RLS, más alto es el valor absoluto de R

¿Cómo calcular la correlación en R?

Función `cor(x,y)`

$x \rightarrow$ predictor

$y \rightarrow$ respuesta

También se puede usar `cor(x)`, donde x es una matriz, para calcular el coeficiente de todas las combinaciones de pares de columna

13.2 Regresión lineal mediante mínimos cuadrados

El objetivo es minimizar la suma de cuadrados de los residuos

Condiciones iniciales:

1. Distribución condicional bivalente entre las variables $\rightarrow$ Para cada valor fijo de X, los valores de Y se distribuyen normalmente con una varianza constante (homocedasticidad)
2. Relación lineal entre la variable X y las medias de la variable Y
3. Observaciones independientes de la muestra

Ajustar la recta de mínimos cuadrados para un par de variables:

`lm(formula, data)`

formula: tiene la forma <variable_respuesta> ~ <variable_predictora>.

data: matriz de datos.

(Script 13.1)

1. Ajuste de un modelo lineal
2. Gráfico del ajuste
3. Predicciones del modelo
4. Gráfico de predicción

Valor $\Pr(<|t|) \rightarrow H_0: \beta_i = 0$ coeficiente tiene valor 0 ($p < 0.05$ rechaza $\rightarrow$ Hay relación)

13.3 Uso del modelo

Es importante analizar el rango de posibles valores de una variable, con el de definir si una extrapolación de datos tendría sentido en un determinado rango de valores

Para predecir valores, podemos reemplazar datos que buscamos predecir en el modelo obtenido

¿En R? Podemos usar `predict(object, newdata)` para predecir respuestas

object: el modelo a emplear.

newdata: una matriz de datos donde exista una columna con el nombre del predictor usado en la fórmula del modelo (para el ejemplo, `disp`) con los nuevos valores para los que se desea efectuar la predicción.

13.4. Regresión lineal con un predictor categórico

La función `lm` ya aplica la transformación de datos categóricos a numéricos

13.5 Confiabilidad de un modelo RLS

13.5.1 Bondad de ajuste

Coeficiente de determinación $R^2 \rightarrow$ Indica la proporción con la que un modelo de RLS reduce la varianza global no explicada

R^2 lo reporta la función `lm()`. Por ej, un R^2 de 0.5 indica que el predictor logra reducir la varianza aleatoria en un 50% respecto al modelo nulo (sin predictores)

De la salida de `lm()`, `F` indica el valor de esa reducción de varianza y entrega un p-valor que si es significativo ($p < 0.05$) indica un buen ajuste de datos y se considera representativo.

13.5.2 Distribución e independencia

Cuando las 3 condiciones iniciales se cumple, podemos ver las siguientes características en los gráficos de residuos:

1. Se distribuyen aleatoriamente en torno a la línea de valor cero
2. Forman una “banda horizontal” en torno al valor cero
3. No hay residuos alejados del patrón
4. No forman patrones reconocibles

Si se cumplen 1,2 y 3, hay distribución condicional normal bivalente entre predictor y salida.
Si se cumplen 1 y 4 es razonable asumir relación lineal y observaciones independientes entre sí

¿Cómo graficar residuos?

`residualPlots(modelo, type="pearson")`

paquete car

modelo: el modelo a emplear.

type: el tipo de residuo a utilizar. Por omisión toma valor "pearson" que corresponde a la definición que hemos visto y graficado anteriormente. Sin embargo, estos residuos se encuentran en la escala de la variable de salida, por lo que a veces es difícil reconocer valores “muy alejados” de la línea del cero. Por esto existen otras opciones, como el tipo "standard", que los presenta normalizados en términos de la desviación estándar presente en los datos, o el tipo "studentized", que los presenta normalizados en términos de la desviación estándar sin considerar el residuo en cuestión. Sabemos que valores fuera del rango ± 2 (que representa aproximadamente el 95% de los datos en distribuciones normales o t) podrían tratarse de valores atípicos.

Decisión

Si el modelo está construido con datos que cumplen las condiciones de distribución e independencia, entonces ninguno de estos gráficos debería mostrar cambios sistemáticos en la distribución de los residuos en el eje y a lo largo del eje x. Por ejemplo, no deberían observarse tendencias no lineales, variación en las tendencias ni puntos aislados.

Una línea recta horizontal en cero, permite asumir la linealidad entre variables

`residual_plots()` incluye una prueba de curvatura donde $\Pr(>|Test\ stat|) > 0.05$ verifica la linealidad (no rechaza H_0). H_0 : linealidad entre variables

Para variables categóricas, aislar

`residualPlots(modelo, terms = ~ variable_numerica)`

Considerar:

1. Si funciona

```
# Cargamos datos y paquetes necesarios
library(car)
data(mtcars)
```

```
# Ajustamos un modelo lineal de manera nativa
modelo_nativo <- lm(mpg ~ wt + hp, data = mtcars)
```

```
# ¡Funciona perfecto! residualPlots reconoce las variables limpias de mtcars
residualPlots(modelo_nativo)
```

2. No funciona

```
# Cargamos caret
library(caret)
library(car)
```

```
# Entrenamos usando train() de caret con remuestreo (boot)
control_enrenamiento <- trainControl(method = "boot", number = 10)
modelo_caret <- train(mpg ~ wt + hp, data = mtcars,
                      method = "lm",
                      trControl = control_enrenamiento)
```

```
# Extraemos el modelo final (aquí es donde se arrastran los artefactos complejos)
modelo_peso_final <- modelo_caret$finalModel
```

```
# ¡Probable Error! residualPlots fallará buscando '.outcome' o el entorno de datos original
residualPlots(modelo_peso_final)
```

¿Cómo evaluar la presencia de autocorrelación entre residuos?

```
durbinWatsonTest(model)
```

paquete car

Requiere de una semilla, ya que aplica bootstrapping

H0: Observaciones independientes (no presentan autocorrelación)

$p > 0.05 \rightarrow$ no hay evidencia para descartar que las obs son independientes

Otra función útil:

```
marginalModelPlots(modelo, sd = FALSE)
```

modelo: el modelo a emplear.

sd: si tiene valor TRUE, marca líneas de desviación estándar estimadas en el gráfico para facilitar la evaluación de los supuestos sobre la varianza

Análisis: Las curvas punteada y del modelo, deben coincidir para probar que el modelo presenta un buen ajuste a los datos

Prueba de varianza del error no constante

`ncvTest(modelo)`

H0: Se cumple la condición de homocedasticidad

$p < 0.05 \rightarrow$ no se cumple H0 (rechazar)

¿Qué ocurre si no se cumple la condición de homocedasticidad?

Opción 1: Asumir que la desviación se debe a la baja cantidad de observaciones y continuar con el modelo

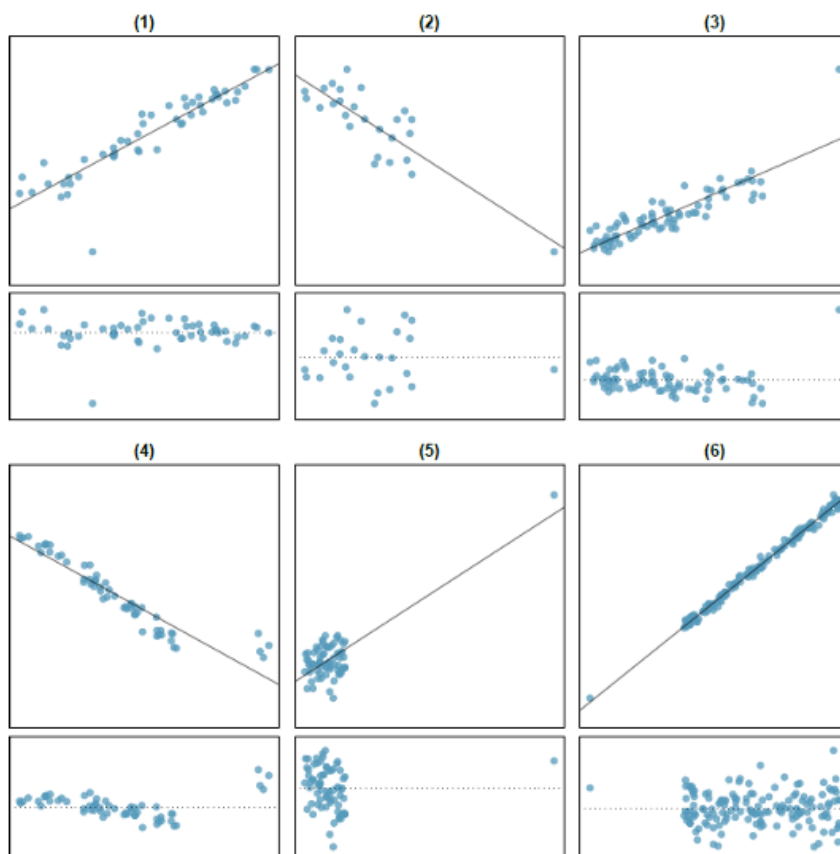
Opción 2: Utilizar métodos de remuestreo para arreglar los datos

Opción 3: Desechar el modelo y buscar un nuevo predictor

(Script 13.3)

1. Gráficos de residuos
2. Autocorrelación
3. Gráficos marginales
4. Varianza del error no constante
5. Gráficos de influencia

13.5.3 Influencia de los valores atípicos



(3), (4) y (5) son puntos influyentes

Apalancamiento o valor hat

Distancia de Cook (D_i): mide el efecto combinado que tiene un punto sobre los coeficientes de regresión del modelo cuando es incluido o excluido del ajuste.

$D_i > 1$ indica influencia indebida

También se usa el umbral $4/n$ con muestras grandes

Valores mucho mayores al resto también son sospechosos

Los valores de distancia de Cook obtenidos con `influencePlot()` no deben superar el umbral

Queda a criterio del investigador si es conveniente quitar o no un dato influyente del modelo

13.6 Calidad predictiva de un modelo RLS

Debemos verificar que las predicciones que genera el modelo son de buena calidad

13.6.1 Error de un modelo RLS

MSE (error cuadrático medio) como métrica para evaluar la calidad del modelo

RMSE → raíz cuadrada de MSE que se expresa en la misma escala que la variable Y.

Indica cuánto se desvían las predicciones de los valores reales de la variable.

13.6.2 Generalización

Diremos que un modelo es generalizable si para un conjunto de datos nuevo consigue predicciones con una calidad similar al que consigue con los datos usados en su construcción.

Validación cruzada

Separa el conjunto de datos en 2 fragmentos:

Conjunto de entrenamiento: Se suele usar entre 70 y 90% de la muestra. Se emplea para ajustar la recta con el método de mínimos cuadrados

Conjunto de prueba: Contiene el 10 a 30% restante. Se usa para evaluar el modelo con los nuevos datos

(Script 13.4) Validación cruzada en regresión lineal simple

$p < 0.05$ → Reducción de la varianza aleatoria es significativa

También es útil que los valores RMSE para cada conjunto sean parecidos (el modelo generaliza bien)

El conjunto de pruebas podría ser no representativo producto del azar, por lo que usamos **validación cruzada de k pliegues**:

Se separa el conjunto en k subconjuntos disjuntos de igual tamaño y obtenemos las estimaciones del error:

1. Para cada k:
 - a) Tomar uno de los k subconjuntos (pliegue) y reservarlo como conjunto de prueba
 - b) Ajustar la recta de mínimos cuadrados
 - c) Estimar MSE usando el conjunto de prueba

¿Cómo aplicar validación cruzada de k-pliegues?

Con la función `train(formula, method="lm", trControl=trainControl(method="cv", number)`

Ojo: Hay valores fijos que permiten usar la función específicamente para un modelo RLS

paquete caret

formula: fórmula que se emplea en las llamadas internas a la función `lm()`.

number: cantidad de pliegues (k).

(requiere una semilla ya que aplica muestreo aleatorio)

RLS

```
library(caret)
```

```
# 1. Configurar validación cruzada de 10 pliegues
control_cv10 <- trainControl(method = "cv", number = 10)
```

```
# 2. Entrenar el modelo RLS
```

```
set.seed(123)
```

```
modelo_qls <- train(
  formula = rm ~ lstat,      # Una sola variable predictora (X)
  data = datos_muestra,
  method = "lm",            # Modelo lineal estándar
  trControl = control_cv10
)
```

```
# Ver resultados (RMSE, R2, MAE)
```

```
print(modelo_qls)
```

RLM

```
library(caret)
```

```
# Configurar control (pueden ser 10 pliegues)
```

```
control_cv10 <- trainControl(method = "cv", number = 10)

# Entrenar el modelo RLM con dos o más predictores
set.seed(123)
modelo_rlm <- train(
  formula = rm ~ lstat + medv + age, # Múltiples predictores en la fórmula
  data = datos_muestra,
  method = "lm",                    # Sigue siendo "lm" para modelos lineales
  trControl = control_cv10
)

# Ver resultados de calidad predictiva promedio en los 10 pliegues
print(modelo_rlm)
```

Logística

```
library(caret)

# Configurar control de validación cruzada
control_cv10 <- trainControl(method = "cv", number = 10)

# Entrenar el modelo de Regresión Logística
set.seed(123)
modelo_logistico <- train(
  formula = EN ~ Weight + Height, # Variable de respuesta binaria (factor) ~ predictores
  data = muestra_total,
  method = "glm",                 # ¡Cambia a "glm" para regresión logística!
  family = binomial,              # Define explícitamente la familia binomial
  trControl = control_cv10
)

# Ver resultados de desempeño predictivo (Accuracy y Kappa globales)
print(modelo_logistico)
```

Resumen de los valores fijos y variables:

- **Lo que se mantiene fijo:** La estructura de `trainControl(method = "cv", number = 10)` y la sintaxis general de la función `train(formula, data, method, trControl)`.
- **Lo que cambia según el modelo:**
 - **Fórmula:** $Y \sim X1$ (Simple) vs $Y \sim X1 + X2 + X3$ (Múltiple).
 - **method:** "lm" para regresiones continuas (RLS y RLM) y "glm" con `family = binomial` para regresión logística binaria.